Processor Design: Pipelining

Adamya Verma *Anup Chavan*
Bhumin Hirpara *Roshan Singh*

05-Sep-2024

Multicycle Datapath

- A single-cycle processor implementation violates the design principle of *making the common case fast*. So, we attempt to break the single cycle datapath into multiple stages to improve the performance.

In a single cycle processor, the clock cycle is equal to the worst-case delay for all instructions

Stages of Multicycle Processor

We divide the datapath into 5 different stages, based on the functionality served by each stage.

1. IF: Instruction fetch from instruction memory
2. ID: Instruction decode (identifying instruction type, source/destination registers, and generation of immediate values) as well as reading source registers
3. EX: Execute required operation or calculate address (ALU operation)
4. MEM: Access memory (read operand or write to memory)
5. WB: Write result back to the register

The following Figure provides an overview of the multicycle datapath and which stages are used by which instructions. Note that the control signals are not fully defined yet.

Instruction	IF	ID	EX	MEM	WB	Cycles
`ld`	Yes	Yes	Yes	Yes	Yes	5
`sd`	Yes	Yes	Yes	Yes		4
R-format	Yes	Yes	Yes		Yes	4
`beq`	Yes	Yes	Yes			3

Table 1: Stages used in each instruction

- A multicycle datapath generally improves performance over a single-cycle datapath by breaking down the execution of instructions into smaller steps.
- The stage that needs the longest delay determines the clock period.

There wouldn't be any difference in the performance if all instructions in a program make use of all cycles. As explained later in the lecture, pipelining improves the performance even more than our current implementation.

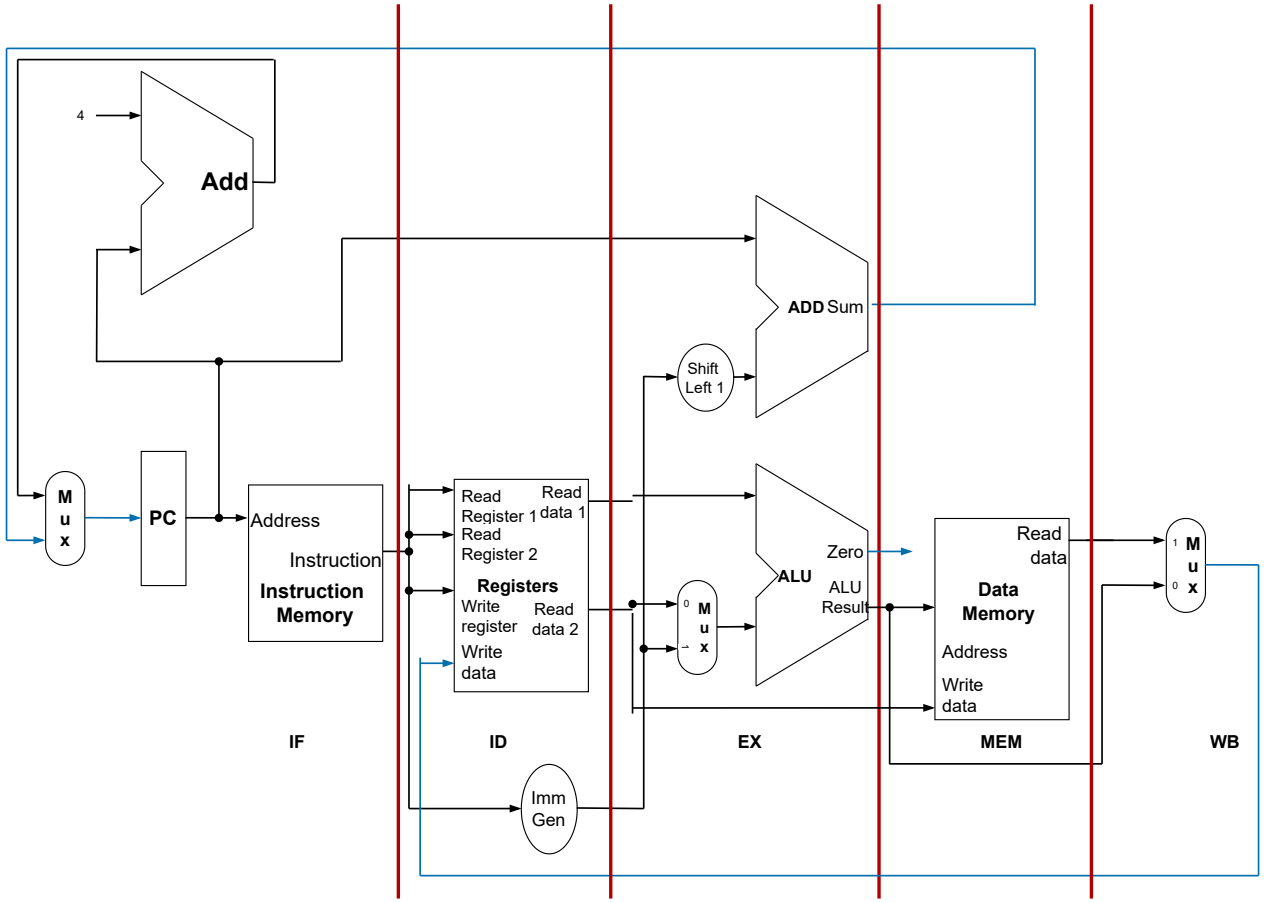


Figure 1: Building a multicycle datapath from a single cycle datapath

- Assuming all stages are of similar delay, the clock period of a multi-cycle datapath will be one-fifth of that of a single-cycle design. The instructions which require less than 5 cycles to complete will take lesser time as compared to a single cycle implementation.
- The control needs to be updated to assert appropriate signals in the corresponding cycle, based on the instruction type. The logic would be similar to single-cycle design, but the sequencing needs to be as per the multi-cycle design.

Pipelining

One of the key issues with the discussed multi-cycle implementation is that when an instruction moves from one stage to another, the previous stage remains idle. In general, only one of the five stages is active at a time and leads to idle resources. The idea of pipelining enables using the available resources for the other instructions. Once an instruction moves from a stage to next, let the next instruction use

that stage.

Pipelining in everyday life

- *Queue in dining hall:* You don't have to wait for the person in front of you to finish getting all their food before you start collecting yours.
- *Traffic flow:* Cars move through different stages of a traffic signal, similar to how instructions move through stages in pipelining.
- *Coming to class:* Different students are at various stages—some are walking, some are parking, and others are already seated. This is similar to how pipelining allows multiple tasks to progress simultaneously, but at different stages.

Imagine the road is narrow, like a single-core processor; while some students walk faster or cycle, they still have to wait for the slower ones, representing pipeline hazards. If the road were wider, allowing multiple lanes, this would be like parallel processing, where faster tasks don't have to wait.

- Pipelining uses a datapath very similar to the single-cycle datapath but is much more efficient by having a much higher throughput.
- In a multicycle implementation with pipelining, each execution step takes one clock cycle, allowing a functional unit to be reused in different cycles within the same instruction.
- Clock period or frequency is determined by the stage with the longest delay.

Comparing Pipelined processor and Single-cycle processor

Consider a scenario where time for stages is 100 ps for register read or write and 200 ps for other stages. Using Table , we can compute the total time required for each instruction:

Instruction	Instruction fetch	Register read	ALU Operation	Memory Access	Register Write	Total Time
`ld`	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
`sd`	200 ps	100 ps	200 ps	200 ps	–	700 ps
R-format	200 ps	100 ps	200 ps	–	100 ps	600 ps
`beq`	200 ps	100 ps	200 ps	–	–	500 ps

Table 2: Instruction Execution Time Breakdown

The clock period must be at least the longest time any instruction can take. In this case, it is the load instruction (highlighted cell).

- If all the stages were balanced, i.e., if all stages took the same time, we have:

$$\text{time delay between 2 instructions}_{\text{pipelined}} = \frac{\text{time delay between 2 instructions}_{\text{non-pipelined}}}{N}$$

where N is the number of stages. Figure explains how different instructions execute with time in single cycle and pipelined processors respectively.

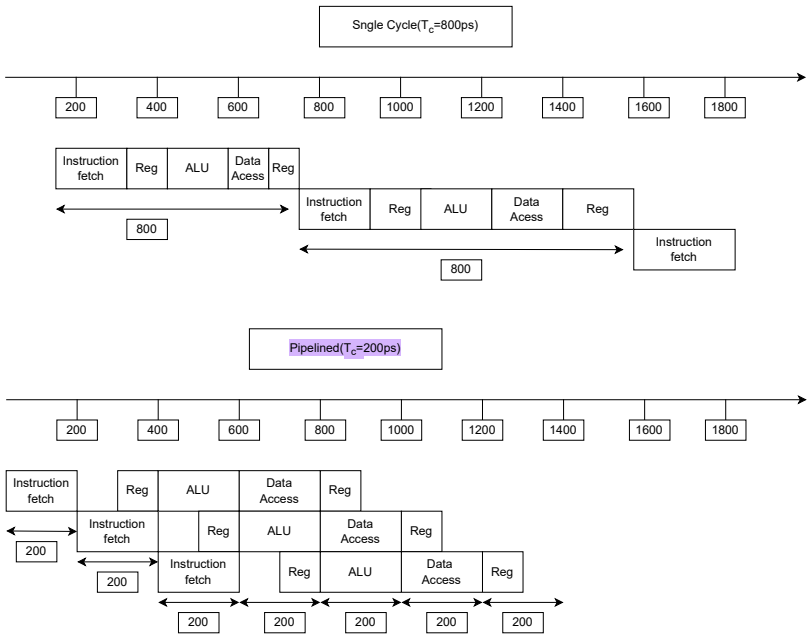


Figure 2: Single cycled vs Pipelined processor

- Since the stages are not balanced in our example, the speedup would be less. (Why? The register write stage would need to wait 100 picoseconds more as the clock period needs to match the delay of the longest stage.)
- Note that each instruction still takes the same amount of time as in a single-cycle processor as each instruction needs to go through all the stages; the performance gain (speedup) comes from increased throughput.
- The latency (time for each instruction) does not decrease. In fact, the latency might increase when stages are not balanced.

Pipeline Speedup

Consider a pipelined processor with p stages, which needs to execute N instructions. Then,

- **Clock period:**
  - for a *single-cycle processor*, the clock period would be $T_s =$ total delay

  - for the *multicycle pipelined processor*, clock period is T_p = Longest stage delay $\approx T_s / p$

- **Total time:**
  - Total time needed for single-cycle processor $= N \times T_s$
  - Total time needed for pipelined processor $\approx \dfrac{N \cdot T_s}{p}$
  - Speedup $\approx p$

To derive this result, note that the 1st instruction completes in p cycles, each one after this is offset by one period T_p

So, total number of cycles = $p + N - 1 \approx N$ for large N.

Total time $\approx N \cdot T_p = \dfrac{N \cdot T_s}{p}$

- Pipelining does not reduce the time it takes to complete an individual instruction, also called the **latency**
- Pipelining increases the number of simultaneously executing instructions (throughput) and the rate at which instructions are started and completed.
- Pipelining is more effective when N is large.

RISC-V ISA helps you simplify hardware design

- In RISC-V, since all instruction types have source and destination registers at fixed positions within the 32-bit instruction, no additional multiplexors (data selectors) are needed to identify them.
- The RISC-V ISA simplifies pipelining compared to CISC architectures like x86, where variable-length instructions complicate implementation.
- Fewer number of instructions simplify decode stage for RISC-V
- However, this doesn't mean RISC is inherently *better* or *faster* than CISC in all cases. CISC architectures offer advantages, such as requiring fewer registers due to memory operands and simplifying certain tasks like adding multiple integers, thanks to flexible instruction lengths.

RISC-V Pipelined Datapath

- Since the value of one stage might change when it starts operating on the next instruction, we need a mechanism to hold the values. To keep the instruction's value (or ALU operation result, etc.) for the next stage, it must be stored between each stage for an instruction which we achieve by adding additional registers in between every adjacent stages. These registers are called pipeline

Washing machine analogy: we need to have a basket between each pair of stages to hold the clothes for the next step.

registers. They hold the values generated from one stage and make them available to the next stage for one clock cycle.

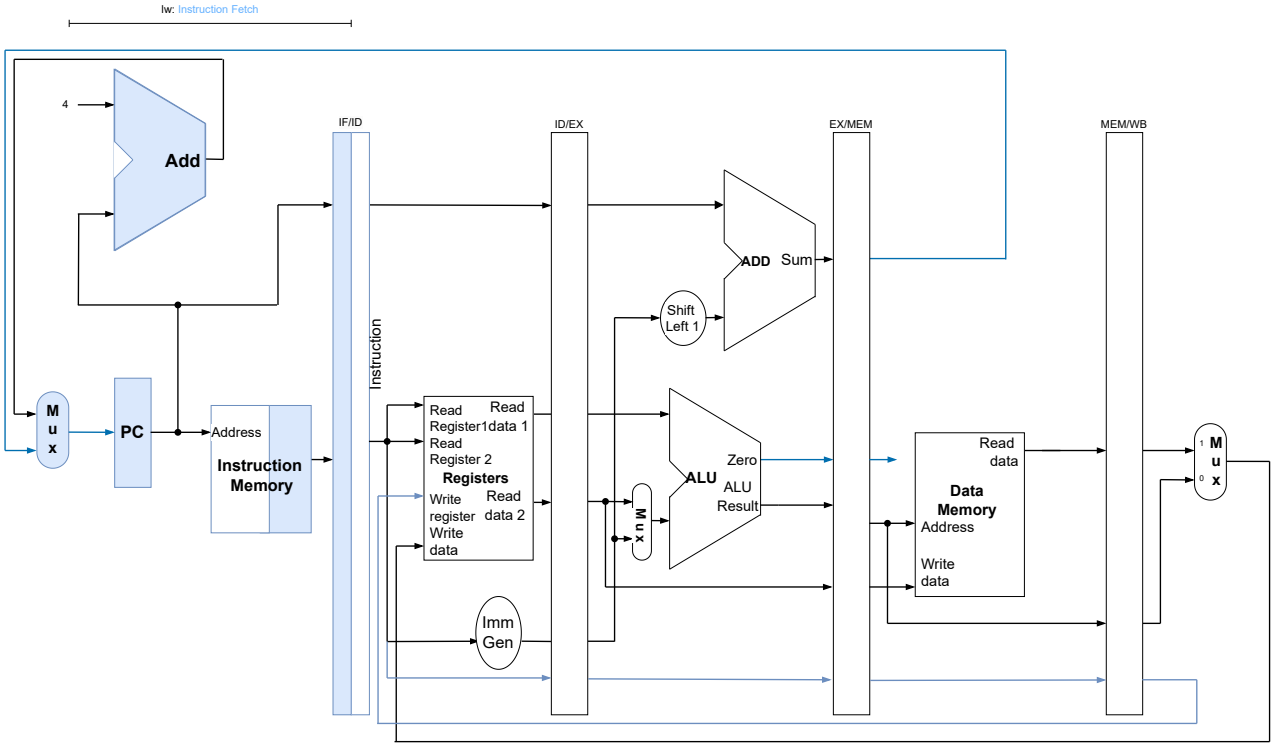


Figure 3: Pipelined version of the datapath in Figure

- All instructions advance during each clock cycle from one pipeline register to the next.
- The registers are named for the two stages separated by that register. For example, the pipeline register between the Instruction Fetch and Instruction Decode stages is called IF/ID or ID/EX, etc. See Figure

Pipelined registers

- Pipelined Registers do not have any other purpose than to support the pipeline.
- Pipelined registers cannot be accessed through the ISA, cannot be read or written to directly through instructions.
- Every instruction updates the PC, whether by incrementing it or by setting it to a branch destination address. The PC can be thought of as a pipeline register: one that feeds the IF stage of the pipeline. However, the PC is part of the visible architectural state; its contents must be saved when an exception occurs, while the contents of the pipeline registers can be discarded.

Example: Sequence of cycles for a load instruction

Convention for highlighting read and write: We highlight the **right half** of registers or memory when they are being **read** and highlight the **left half** when they are being **written**.

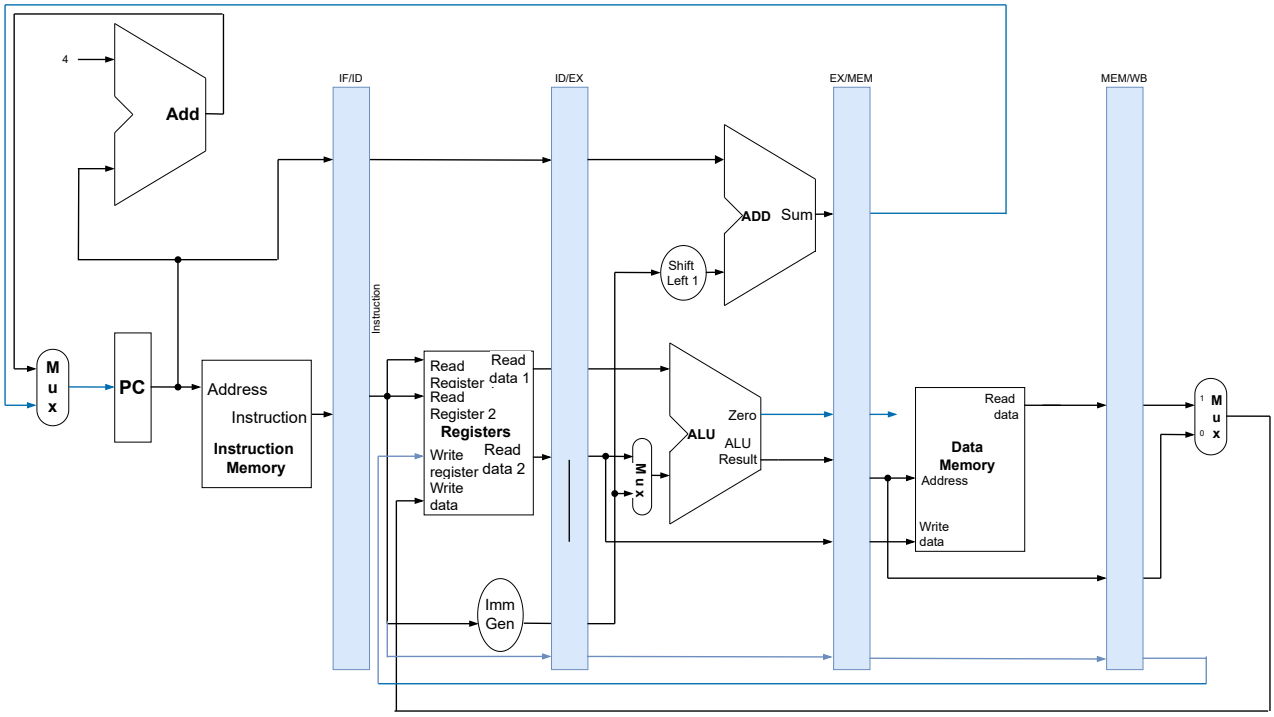


Figure 4: *Instruction Fetch*: The instruction is read from memory using the address in the PC and then is placed in the IF/ID pipeline register

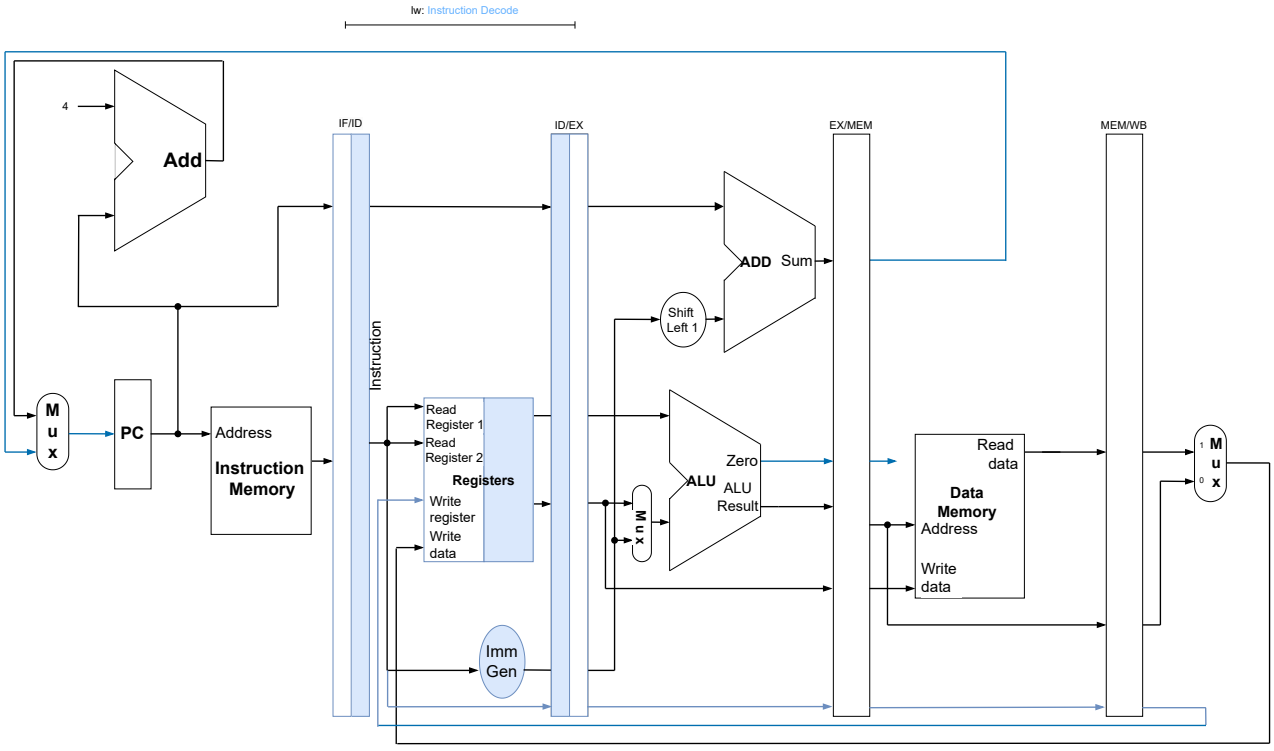


Figure 5: *Instruction decode and register file read*: The IF/ID pipeline register provides the register numbers to read two registers and extends the immediate operand. These three 64-bit values are stored in the ID/EX pipeline register.

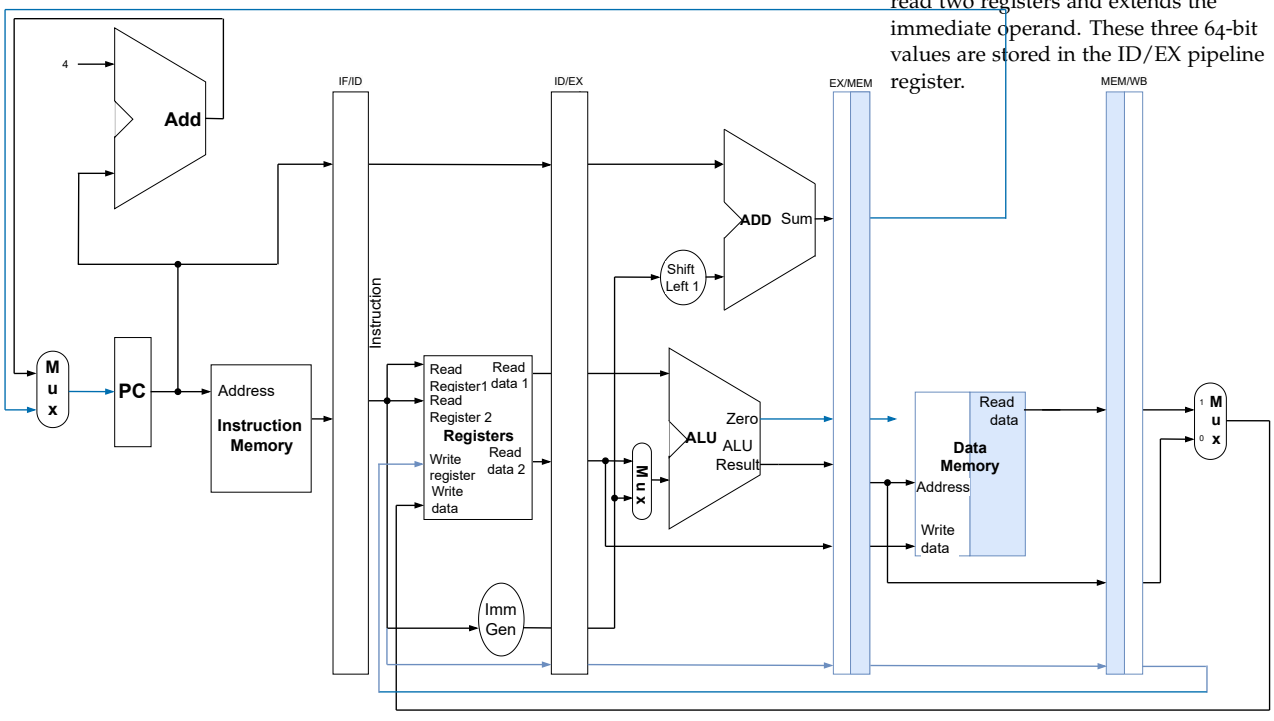


Figure 6: *Execute and address calculation*: the effective address is placed in the EX/MEM pipeline register.

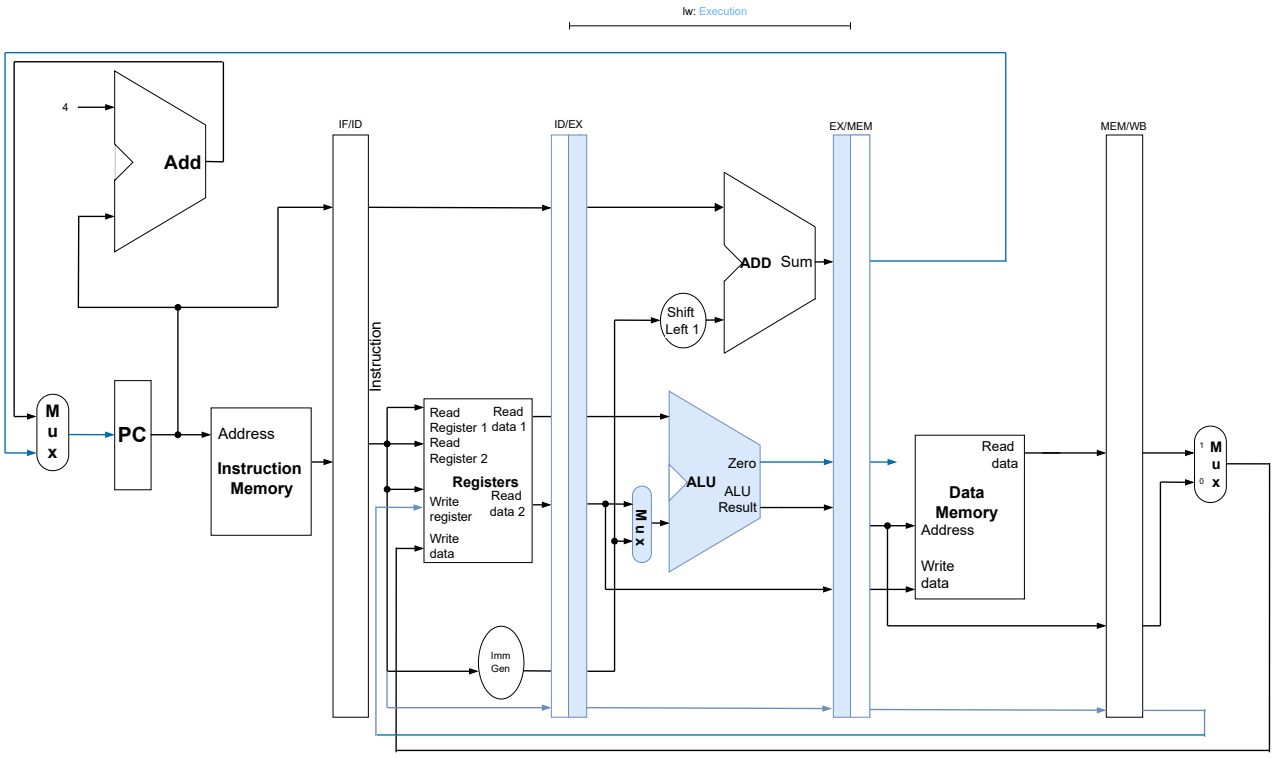


Figure 7: *Memory access*: Data memory is read using the address in the EX/MEM pipeline registers, and the data are placed in the MEM/WB pipeline register.

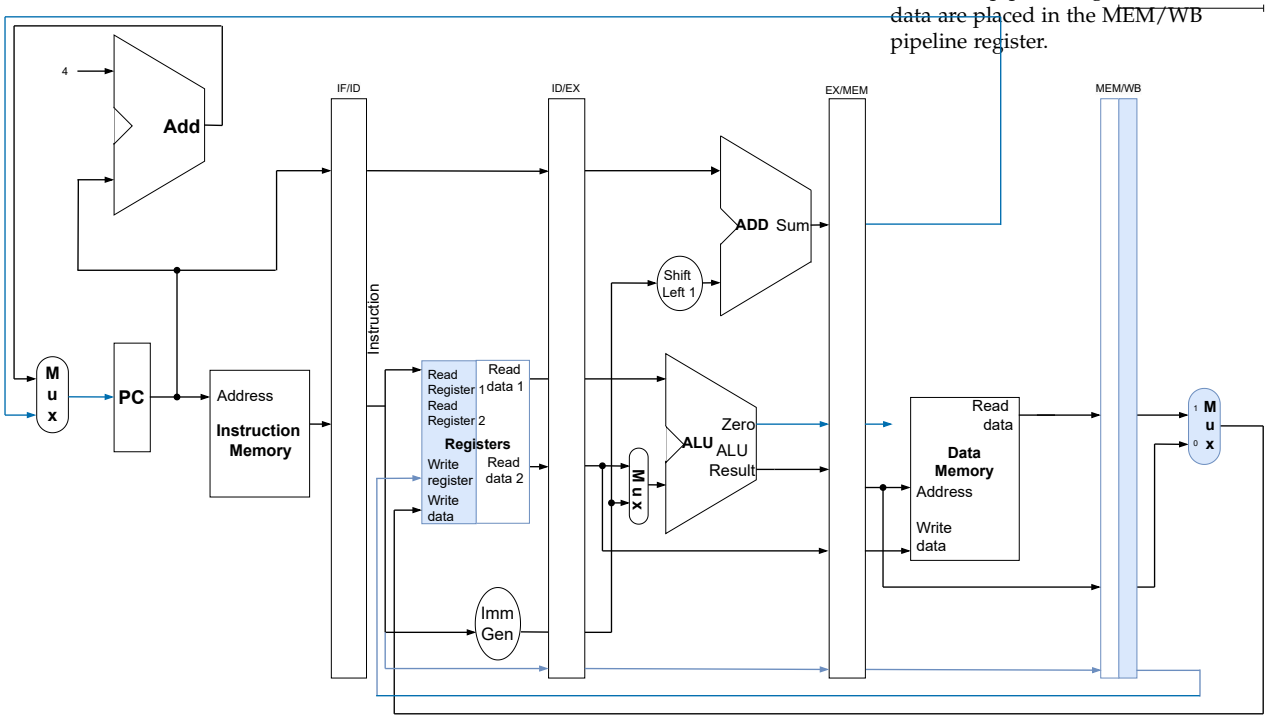


Figure 8: *Write Back*: Data memory is read using the address in the EX/MEM pipeline registers, and the data are placed in the MEM/WB pipeline register.